API Documentation
Book Management API + Authentication

Backend .NET Internship — Week 3 Day 5 / Week 4 Day 1
Prepared by: Mithgal Alhrene | Updated: 09-08-2026

Purpose
A practical reference for developers who need to understand the available endpoints, request format, and expected HTTP responses without opening the Postman collection.

1. API Overview

Method	Endpoint	Purpose
GET	{{baseUrl}}/api/books	Retrieve all books
GET	{{baseUrl}}/api/books/{id}	Retrieve a book by ID
POST	{{baseUrl}}/api/books	Create a new book
PUT	{{baseUrl}}/api/books/{id}	Update an existing book
DELETE	{{baseUrl}}/api/books/{id}	Delete a book
POST	{{baseUrl}}/api/Auth/register	Register a new user

2. Base URL & Environment

All requests use the Postman Environment variable **{{baseUrl}}**. Select the correct environment in Postman before sending requests.

3. Book Endpoints

GET	GET_ALL

URL: {{baseUrl}}/api/books

Purpose: Retrieve all books.

Request Body: None required.

Success: 200 OK

GET	GET_By_Id

URL: {{baseUrl}}/api/books/1

Purpose: Retrieve a specific book by its ID.

Request Body: None required.

Success: 200 OK

Possible Error: 404 Not Found

GET GET_By_Id_NotFound

URL: {{baseUrl}}/api/books/99999

Purpose: Test retrieval using an ID that does not exist.

Request Body: None required.

Success: 404 Not Found

POST POST_Create

URL: {{baseUrl}}/api/books

Purpose: Create a new book.

Request Body:

```
{ "bookName": "Clean Code", "authorId": 1, "publishedYear": 2008, "price": 49.99 }
```

Success: 201 Created

POST POST_Create_Invalid

URL: {{baseUrl}}/api/books

Purpose: Test the create operation with an empty request body.

Request Body:

```
{}
```

Success: 500 Internal Server Error

PUT PUT_Update

URL: {{baseUrl}}/api/books/1

Purpose: Update an existing book.

Request Body:

```
{ "bookId": 1, "bookName": "Clean Code (2nd Edition)", "authorId": 1, "publishedYear": 2020,
"price": 59.99 }
```

Success: 200 OK

PUT PUT_Update_NotFound

URL: {{baseUrl}}/api/books/99999

Purpose: Test updating a book using an ID that does not exist.

Request Body:

```
{ "bookId": 99999, "bookName": "Unknown Book", "authorId": 1, "publishedYear": 2020, "price":
10.00 }
```

Success: 404 Not Found

PUT PUT_Update_Invalid_Id

URL: {{baseUrl}}/api/books/abc

Purpose: Test the update operation using a non-numeric ID.

Request Body:

```
{ "bookName": "Invalid", "authorId": 1, "publishedYear": 2020, "price": 10 }
```

Success: 400 Bad Request

DELETE DELETE

URL: {{baseUrl}}/api/books/1

Purpose: Delete a book by its ID.

Request Body: None required.

Success: 204 No Content

DELETE DELETE_NotFound

URL: {{baseUrl}}/api/books/99999

Purpose: Test deletion using an ID that does not exist.

Request Body: None required.

Success: 404 Not Found

4. Authentication Endpoint

The authentication section currently contains a user registration endpoint. It accepts an email and password and returns a successful registration response when the supplied credentials pass the configured validation and the user is not already registered.

POST	Register

URL: {{baseUrl}}/api/Auth/register

Purpose: Register a new user.

Request Body:

```
{
"email": "mithgal@gmail.com",
"password": "Mithgal123"
}
```

Success: 200 OK — { "message": "User registered successfully" }

Possible Error: 400 Bad Request for validation errors or when the username/email is already taken.

5. Register Validation Examples

Validation / Error Code	Meaning
PasswordRequiresNonAlphanumeric	Password must have at least one non-alphanumeric character.
PasswordRequiresUpper	Password must have at least one uppercase character (A-Z).
DuplicateUserName	The supplied username/email is already taken.

Example valid password: `Mithgal123!`

6. Testing Summary

Book API — Happy Path: GET_ALL (200 OK), GET_By_Id (200 OK), POST_Create (201 Created), PUT_Update (200 OK), and DELETE (204 No Content).

Book API — Error Path: GET_By_Id_NotFound (404), PUT_Update_NotFound (404), PUT_Update_Invalid_Id (400), DELETE_NotFound (404), and POST_Create_Invalid (500 Internal Server Error).

Authentication: Register was tested with a valid request (200 OK), while validation and duplicate-user scenarios returned 400 Bad Request with the corresponding error codes.

7. Postman Usage

1. Select the required request from the Postman collection.
2. Make sure the environment containing **{{baseUrl}}** is selected.
3. For POST and PUT requests, enter the required JSON request body.
4. Send the request and compare the returned HTTP status with the documented result.
5. For ID-based endpoints, replace the example ID with the target book ID when needed.

8. Conclusion

The API provides the core CRUD operations for book management and now includes user registration. This document gives developers a concise reference for the available endpoints, request formats, and expected HTTP responses.